

Checkpoint — Model Retraining & Airflow Integration

Yassine Ben Jemia — Amen Bank DCSI

July 22–23, 2026

1 Session Objectives

Retrain both anomaly detection models (Autoencoder, LSTM) on the aligned 30-feature schema produced by `EnrichmentCore._compute_features()`, wire the retraining pipeline into the Airflow `weekly_retrain` DAG via `DockerOperator`, and validate end-to-end streaming pipeline output with the new models.

2 Model Retraining on 30-Feature Schema

2.1 Pipeline

The retraining data pipeline follows the path:

```
PostgreSQL → batch_enrichment.py → training_features.csv → data_prep.py →  
train_all.py
```

Key property: `batch_enrichment.py` calls the same `_compute_features()` function used in streaming, guaranteeing zero feature drift between training and inference.

2.2 Changes Applied

- Added `timestamp` to `batch_enrichment.py` output columns (required by `data_prep.py` for temporal split)
- Created `src/training/train_all.py` — unified entry point: single `prepare()` call, trains AE then LSTM sequentially, computes and saves reference score distributions (`ae_ref_scores.npy`, `lstm_ref_scores.npy`) for percentile normalization
- Moved training runners into `src/training/`
- Removed `difficulty` grouping from `evaluate()` in both model modules (column not persisted past generator CSV)
- Extracted `compute_scores()` from `evaluate()` in both modules for reference distribution computation
- Added `./data:/app/data` and `./src:/app/src` volume mounts across all `Dockerfile.api` services for dev convenience
- Added DB environment variables to `enrichment/scoring/decision` services

2.3 Results

AUC decrease is expected — 30 features carry less redundant information than 74. LSTM lost more because sequence modeling benefits from richer per-step representations. Both models remain strong for production use.

Model	Old AUC (74 feat.)	New AUC (30 feat.)	Delta
Autoencoder	0.9853	0.9714	−0.0139
LSTM	0.9866	0.9355	−0.0511

Table 1: Test AUC comparison: 74-feature vs 30-feature schema

Scenario	AUC	Test Count
smurfing	1.0000	54
cheque_structuring	0.9930	41
timing_anomaly	0.9795	3
frequency_burst	0.9579	34
duplicate_virement	0.9272	10
cumulative_threshold	0.6799	5
amount_spike	0.5776	2

Table 2: Per-scenario AUC — AE on 30-feature test set

2.4 Per-Scenario AUC (Autoencoder, 30 features)

3 Airflow Integration via DockerOperator

3.1 Architecture

The Airflow scheduler does not have PyTorch installed (by design — lightweight orchestrator). Training runs in a dedicated Docker container triggered by `DockerOperator`:

```
Airflow Scheduler Dockersocket amen-anomaly-training:latest → train_all.py
```

3.2 Changes Applied

- Added `apache-airflow-providers-docker` to `requirements-airflow.txt`
- Mounted Docker socket (`/var/run/docker.sock`) into Airflow scheduler and webserver
- Pinned training image name: `image: amen-anomaly-training:latest` in `docker-compose.yml`
- Replaced placeholder `retrain_models` task in `weekly_retrain` DAG with `DockerOperator`
- Set `force_pull=False` (local image, not on Docker Hub) and `mount_tmp_dir=False` (host Docker daemon incompatibility)

3.3 Validation

DAG triggered manually via CLI. `DockerOperator` successfully:

- Started container from `amen-anomaly-training:latest`
- Ran full training pipeline (AE + LSTM)
- Saved models and reference distributions to `models/` via bind mount
- Exited cleanly — task marked SUCCESS in Airflow

4 Streaming Pipeline Validation

4.1 Cold-Start Issue Identified

End-to-end streaming tests revealed systematically inflated scores (avg `fused_score` = 0.99). Root cause diagnosed through progressive elimination:

1. **Reference distributions** — verified correct (141k AE, 98k LSTM scores, proper range)

2. **Buffer hydration** — Redis buffers confirmed populated (18,817 keys)
3. **Temporal mismatch** — hydrated buffers contained future events relative to simulator offset, causing velocity features to explode
4. **Root cause confirmed:** Zero-velocity feature vectors (cold-start) produce AE reconstruction error of 0.8655, mapping to percentile 0.9983. The model never saw these patterns because training applied a 21-day warmup filter.

4.2 Partial Fixes Applied

- **Enrichment core:** `has_duplicate_recent` guarded — returns 0 when `len(cbuf) < 5`
- **Decision engine:** Rule 8 (frequency burst) replaced with Poisson z-score threshold: $threshold = \mu + 3\sqrt{\mu}$, where $\mu = \text{expected_daily_rate}$. Adapts to client archetype instead of flat 5x multiplier.
- **Decision core:** `expected_daily_rate` floor raised from 0.1 to 1.0

4.3 Remaining Fix (Next Session)

Cold-start scoring guard: Add minimum buffer check in `ScoringCore` — if client buffer has fewer than N events, return neutral score (0.5) instead of running the model. Same principle as LSTM's `seq_len` gate.

5 Infrastructure Improvements

- `./src:/app/src` volume mount on all API services — code changes reflected without image rebuild
- `./data:/app/data` mount on enrichment service — enables batch CSV output
- DB environment variables added to enrichment, scoring, and decision services
- Training container uses `profiles: ["training"]` — on-demand only

6 Next Session Agenda

1. Implement cold-start scoring guard in `ScoringCore`
2. Re-validate streaming pipeline with guard active
3. Proceed to remaining roadmap items: DLQ, structured logging, MLflow
4. Begin rapport de stage drafting